MASTER 2
ÉCONOMÉTRIE ET STATISTIQUES

Parcours EQUADE

GRANDE ÉCOLE D'ACTUARIAT ET DE GESTION DES RISQUES

INSTITUT DE SCIENCE FINANCIÈRE ET D'ASSURANCES

Rapport de TD1 Data Mining Avancé

Architecture Distribuée sous R

Étude Comparative des Stratégies de Parallélisation pour le Clustering

Résumé : *Ce rapport présente la mise en œuvre d'architectures de calcul parallèle sous R, appliquées à l'optimisation de l'algorithme des K-Means. Après la configuration de l'environnement serveur (SSH, Git, Tmux), nous comparons les performances d'exécution séquentielle face aux approches parallélisées (via les héritages* `multicore` *et* `snow`*). L'étude analyse l'impact du nombre de cœurs et des stratégies de répartition des tâches (chunk size) sur les temps de calcul, tout en validant une méthode d'agrégation pour sélectionner le partitionnement optimal.*

Réalisé par :

ASSOUMANA KALME ANZA
Abdoul Barik

Chargé de cours :

M. Denis Clot

Année Universitaire 2025 - 2026

Table des matières

Introduction **2**

1 Architecture Technique et Environnement **3**
1.1 Procédure de Connexion sur MobaXterm . . . 3
1.1.1 Protocole SSH et Authentification par Clés Asymétriques . . . 3
1.2 Gestion de version avec Git et GitLab . . . 4
1.2.1 Configuration et initialisation . . . 4
1.2.2 Authentification SSH pour GitLab . . . 5
1.2.3 Liaison avec le dépôt distant . . . 5
1.3 Persistance des sessions avec Tmux . . . 6
1.3.1 Gestion des sessions . . . 6
1.3.2 Division de l'écran et navigation . . . 6
1.3.3 Détachement et persistance . . . 6
1.3.4 Cas pratique : Test de persistance . . . 7
1.3.5 Aide-mémoire : Commandes tmux essentielles . . . 7

2 Analyse Exploratoire et Prétraitement **8**
2.1 Distribution de l'activité par Type . . . 8
2.1.1 Chargement et préparation des données . . . 8
2.1.2 Analyse des données brutes . . . 8
2.1.3 Transformation logarithmique . . . 10

3 Analyse de Référence : Approche Séquentielle **12**
3.1 Mesure de performance et impact du découpage . . . 12
3.1.1 Préparation des données pour K-Means . . . 12
3.1.2 Comparaison : Approche directe vs découpée . . . 12
3.1.3 Analyse de performance avec Rprof . . . 14
3.2 Stratégies d'Agrégation des Résultats . . . 14
3.3 Parallélisation locale : Approche Multicore . . . 16
3.4 Parallélisation sur Cluster (Héritage Snow) . . . 18
3.5 Optimisation fine : Load Balancing et Scheduling . . . 19

Conclusion **22**

Introduction

L'augmentation exponentielle du volume de données et la complexité croissante des algorithmes d'apprentissage statistique imposent aujourd'hui de repenser les paradigmes de calcul traditionnels. Bien que plébiscité par la communauté statistique, le langage R exécute nativement ses instructions de manière séquentielle sur un unique cœur logique, limitant de facto son passage à l'échelle face aux exigences du *Big Data*.

L'objectif de ce rapport est d'étudier et de déployer des architectures de calcul distribué et parallèle sous R. Nous nous appuierons sur l'optimisation de l'algorithme de partitionnement des *K-Means* comme cas d'étude. Ce dernier, particulièrement coûteux sur le plan computationnel lors de ses phases de multi-initialisation, constitue un candidat idéal pour l'implémentation de processus de type *embarrassingly parallel* (intrinsèquement parallèles).

Notre démarche analytique s'articule autour de trois axes fondamentaux :

- **L'ingénierie de l'environnement serveur :** Configuration d'une infrastructure sécurisée (SSH), implémentation du contrôle de version (Git/GitLab) et garantie de la persistance des processus (Tmux).
- **L'analyse exploratoire et le prétraitement :** Modélisation des données hospitalières et validation statistique des transformations (notamment logarithmique) au travers du rapport de corrélation η^2.
- **L'évaluation des stratégies de calcul :** Comparaison systématique de l'approche séquentielle avec les environnements `multicore` (mémoire partagée) et `snow` (sockets réseau), incluant une étude de sensibilité sur la granularité des tâches et l'équilibrage de charge (*Load Balancing*).

In fine, cette étude visera à déterminer le paramétrage optimal garantissant le meilleur compromis entre temps d'exécution et coût des communications réseau, afin d'exploiter pleinement le potentiel de l'architecture distribuée mise à notre disposition.

Convention de couleurs : **Git / GitLab** – gestion de version **SSH / Système** – administration **R** – scripts algorithmiques **Cluster** – ressources parallèles **Console** – sorties d'exécution

Note sur la reproductibilité : Les indicateurs de performance (benchmarks) sont tributaires de la charge instantanée du serveur. En raison de la stochasticité des *K-Means* et des cycles de recompilation, de légères variations peuvent apparaître entre les graphiques et l'analyse. Cependant, les **tendances structurelles** demeurent constantes et reproductibles.

Assistance IA : Conception des schémas TikZ et optimisation de la structure LaTeX (Claude, Gemini).

---

1 | Architecture Technique et Environnement

1.1 Procédure de Connexion sur MobaXterm

Contexte et enjeux

Dans le cadre de nos travaux, l'accès aux ressources de calcul intensif du serveur `pages.isfa.fr` est indispensable. La connexion s'effectue à distance, depuis n'importe quelle machine, via une infrastructure sécurisée reposant sur le protocole **SSH (Secure Shell)**.

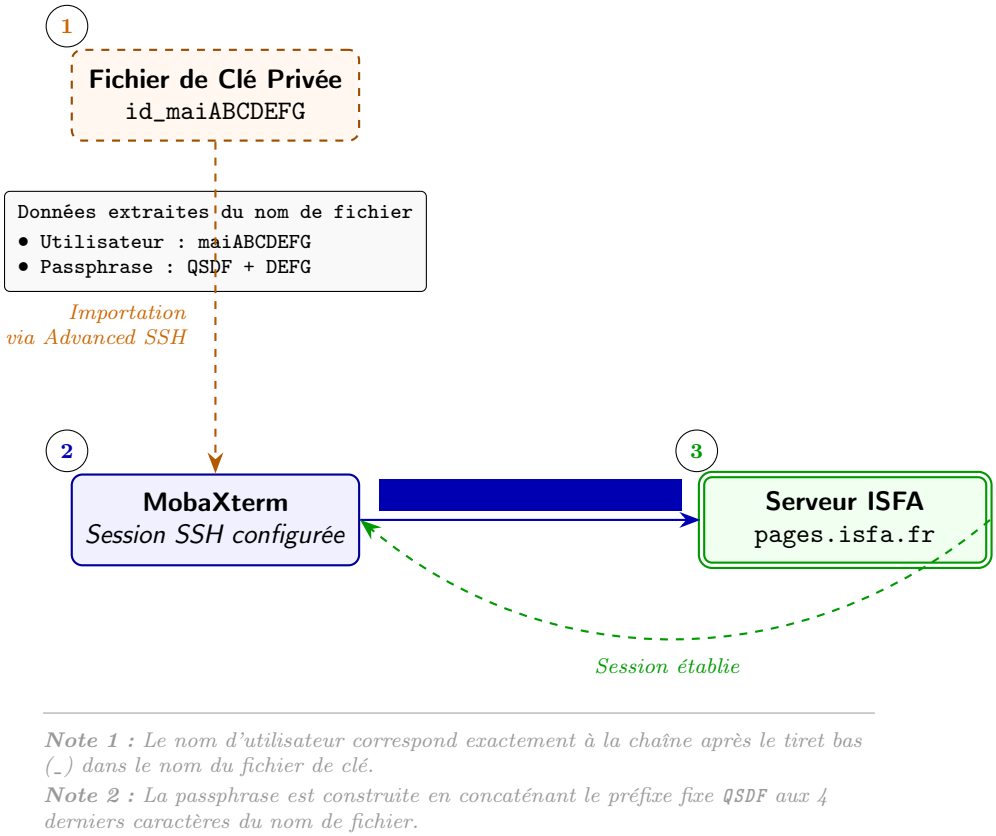


FIGURE 1.1 – Architecture de connexion sécurisée SSH au serveur ISFA via MobaXterm

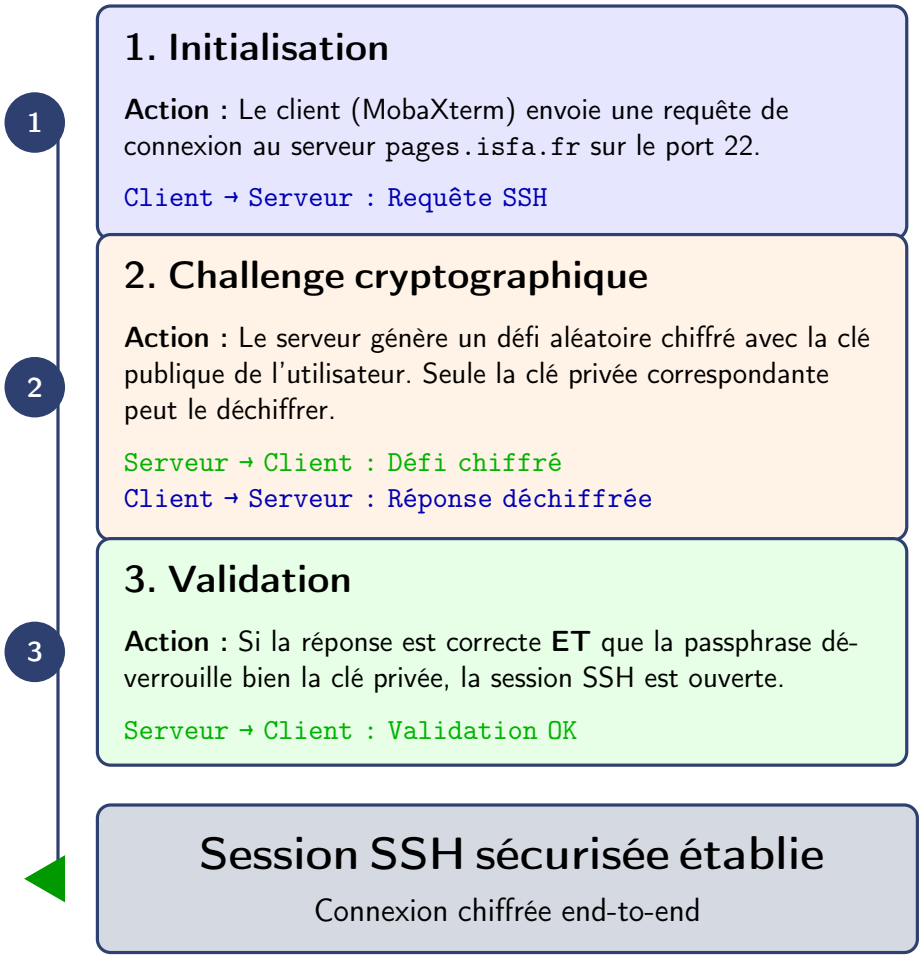
1.1.1 Protocole SSH et Authentification par Clés Asymétriques

La connexion au serveur est établie via le protocole **SSH (Secure Shell)**, standard de l'industrie pour l'administration distante sécurisée. À la différence d'une authentification classique reposant sur un mot de passe vulnérable aux attaques par dictionnaire ou par force brute nous avons mis en place un mécanisme d'**authentification par paires de clés asymétriques**.

Ce mécanisme repose sur deux composantes complémentaires :

— **Clé privée :** Conservée exclusivement sur la machine locale de l'utilisateur (fichier `id_maiABCDEFG`). Elle joue le rôle de *signature numérique unique* et ne transite jamais sur le réseau. Elle est protégée par une passphrase.

— **Clé publique :** Déposée sur le serveur distant, dans le fichier `~/.ssh/authorized_keys`. Le serveur s'en sert pour vérifier l'authenticité de la clé privée présentée lors de la connexion.



Sécurité : *Même l'interception du trafic réseau ne compromet pas le compte. La clé privée ne transite jamais sur le réseau et la passphrase ne quitte jamais la machine locale.*

FIGURE 1.2 – Processus d'authentification SSH en trois phases

Cette architecture garantit la sécurité de la connexion au serveur. Pour les utilisateurs sous Windows (ce qui est notre cas), l'outil **MobaXterm** a été retenu pour sa gestion native des clés SSH, son terminal Unix intégré et son client SFTP permettant le transfert de fichiers.

1.2 Gestion de version avec Git et GitLab

La maîtrise des outils de versionning est essentielle pour assurer la traçabilité du code et faciliter la collaboration. Git permet de suivre l'évolution du projet, tandis que GitLab assure la sauvegarde distante et le partage sécurisé du travail.

1.2.1 Configuration et initialisation

Configuration de l'identité Git

```
git config --global user.email "abdoul-barik...@etu.univ-lyon1.fr"
git config --global user.name "ASSOUMANA KALME ANZA ABDOUL BARIK"
```

Explication : Ces paramètres définissent l'auteur des commits. L'option `--global` les applique à tous les dépôts de l'utilisateur sur cette machine.

Initialisation du dépôt et premier commit

```
cd ~/DMA/TD1
git init

# Creer .gitignore avec : .Rhist* et *.pdf
# Copier hop.csv et creer essai.R

git add hop.csv essai.R .gitignore
git commit -m "Initial commit: donnees et script"
```

Explication : La commande `git init` crée un dépôt dans le répertoire courant. Après avoir créé un fichier `.gitignore` (pour exclure les PDF et historiques R), nous ajoutons les fichiers de travail et réalisons le premier commit.

1.2.2 Authentification SSH pour GitLab

Génération de la paire de clés SSH

```
cd ~/.ssh
ssh-keygen -t rsa -b 4096 -C "clef gitlab pour DMA" -f gitlab_rsa

# Demarrer l'agent SSH et charger la cle
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/gitlab_rsa
```

Explication : Cette commande crée une paire de clés RSA 4096 bits. La clé privée (`gitlab_rsa`) reste secrète sur le serveur, tandis que la clé publique (`gitlab_rsa.pub`) sera déposée sur GitLab. Une passphrase protège la clé privée.

Sur le site **GitLab.com**, après création du compte, nous copions le contenu de `gitlab_rsa.pub` dans la section **Préférences → Clés SSH**.

Récupération de la clé publique

```
cat ~/.ssh/gitlab_rsa.pub
# Copier la sortie (ligne commencant par ssh-rsa...)
```

Explication : La commande `cat` affiche le contenu de la clé publique à copier intégralement dans l'interface GitLab.

1.2.3 Liaison avec le dépôt distant

Après avoir créé un projet vierge `DMA-TD1` sur GitLab, nous lions notre dépôt local au dépôt distant.

Liaison et premier push

```
cd ~/DMA/TD1
git remote add ssh-origin git@gitlab.com:username/dma-td1.git
git push --set-upstream ssh-origin main

# Par la suite, un simple 'git push' suffira
```

Explication : La commande `git remote add` crée un lien nommé `ssh-origin` vers GitLab (URL SSH : `git@gitlab.com:username/dma-td1.git`). Le premier `push` envoie notre branche principale vers le serveur distant.

Note importante : On a utilisé l'URL SSH (`git@gitlab.com:...`) plutôt que HTTPS pour éviter de saisir identifiants et mot de passe à chaque push. En cas d'erreur, on corrige avec : `git remote set-url ssh-origin git@gitlab.com:username/dma-td1.git`

1.3 Persistance des sessions avec Tmux

Lorsque l'on travaille sur un serveur distant, la déconnexion SSH interrompt tous les processus en cours. **Tmux (Terminal Multiplexer)** résout ce problème en créant des sessions persistantes qui continuent à s'exécuter en arrière-plan, même après déconnexion. Tmux permet également de diviser l'écran en plusieurs volets pour travailler simultanément sur plusieurs tâches.

1.3.1 Gestion des sessions

Création et reconnexion à une session

```
# Creer une nouvelle session
tmux new -s dma-td1

# Lister les sessions actives
tmux ls

# Se reconnecter a une session existante
tmux attach -t dma-td1
```

Explication : La commande `tmux new -s nom` crée une session nommée. Pour lister les sessions actives : `tmux ls`. Pour se reconnecter après déconnexion : `tmux attach -t nom`.

1.3.2 Division de l'écran et navigation

Division de la fenêtre en volets

```
# Dans tmux :
Ctrl+b puis "          # Diviser horizontalement
Ctrl+b puis fleches    # Naviguer entre volets
```

Explication : Dans tmux, les commandes se déclenchent via le préfixe `Ctrl+b`. La combinaison `Ctrl+b "` divise la fenêtre horizontalement. Les flèches directionnelles permettent de naviguer entre les volets.

1.3.3 Détachement et persistance

Détachement d'une session

```
# Dans tmux :
Ctrl+b puis d          # Detacher la session

# Apres reconnexion SSH :
tmux attach -t dma-td1 # Retrouver la session
```

Explication : La combinaison `Ctrl+b d` quitte l'affichage de la session sans l'arrêter. Les processus continuent en arrière-plan. Après reconnexion SSH au serveur, on peut réattacher la session pour retrouver exactement l'état précédent.

1.3.4 Cas pratique : Test de persistance

Pour valider la persistance de tmux, nous avons créé une session divisée en deux volets : l'un pour éditer le script `essai.R` avec nano, l'autre pour exécuter R. Dans R, nous avons lancé une boucle infinie affichant un compteur toutes les secondes.

Boucle infinie R pour tester la persistance

```
# Dans R :
i <- 0
while (TRUE) {
    cat(i, "ZZZzzz\n")
    Sys.sleep(1)
    i <- i + 1
}
```

Explication : Cette commande R affiche un compteur incrémental toutes les secondes, simulant un calcul long. L'interruption se fait via `Ctrl+C`.

Après avoir détaché la session (`Ctrl+b d`), nous nous sommes déconnectés du serveur, avons attendu une minute, puis nous sommes reconnectés et avons réattaché la session. Le compteur affichait alors des valeurs élevées (ex : 42, 43...), prouvant que le processus R n'avait jamais été interrompu. Cette fonctionnalité est cruciale pour les calculs de parallélisation qui peuvent durer plusieurs heures.

Pour exécuter notre script complet, nous avons utilisé la fonction `source()` de R.

Exécution d'un script R avec source()

```
# Dans R :
source("essai.R", echo = TRUE, print.eval = TRUE)
```

Explication : La fonction `source()` exécute un fichier script ligne par ligne. Les options `echo=TRUE` et `print.eval=TRUE` affichent les commandes et leurs résultats dans la console.

Cette commande a exécuté l'ensemble des opérations : chargement des données, calculs statistiques, génération des graphiques et export vers le répertoire `~/Sites`.

1.3.5 Aide-mémoire : Commandes tmux essentielles

Le tableau ci-dessous résume les commandes tmux les plus utilisées dans le cadre de ce TD.

liumBlue!20 **Commande**	**Action**
`tmux ls`	Lister toutes les sessions actives
`tmux new -s nom`	Créer une nouvelle session nommée
`tmux attach -t nom`	Se reconnecter à une session existante
`tmux kill-session -t nom`	Supprimer une session (arrête tous les processus)
`Ctrl+b "`	Diviser la fenêtre horizontalement (haut/bas)
`Ctrl+b %`	Diviser la fenêtre verticalement (gauche/droite)
`Ctrl+b` + flèches	Naviguer entre les volets
`Ctrl+b d`	Détacher la session (elle continue en arrière-plan)
`Ctrl+b x`	Fermer le volet actif
`Ctrl+b &`	Fermer la fenêtre active (confirmation demandée)

Table 1.1 – Récapitulatif des commandes tmux utilisées

2 | Analyse Exploratoire et Prétraitement

2.1 Distribution de l'activité par Type

Avant de procéder aux calculs intensifs (K-Means), nous analysons la variable d'intérêt `NbAct` (nombre d'activités) en fonction du `TYPE` d'établissement. Cette étape permet de valider la pertinence d'une transformation des données.

2.1.1 Chargement et préparation des données

La première étape consiste à charger le fichier de données `hop.csv` et à créer la variable `NbAct` correspondant à la somme des activités par ligne (établissement).

Chargement des données et calcul de NbAct

```
# Chargement des donnees
read.csv("hop.csv", stringsAsFactors = TRUE) -> hop

# Calcul de la somme par ligne (toutes colonnes numeriques)
hop$NbAct <- rowSums(hop[sapply(hop, is.numeric)], na.rm = TRUE)

# Verification
summary(hop)
dim(hop)
```

Explication : `read.csv` charge les données en convertissant les variables catégorielles en facteurs. La fonction `rowSums` calcule la somme de toutes les colonnes numériques pour chaque ligne.

Résultat : Dimensions et aperçu des données

```
> dim(hop)
[1] 1539   24

> head(hop)
  TYPE AA AB AC  AD  AE AF  AG AH AI AJ AK AL AM AN AO AP AQ AR AS AU  AV   AX AZ NbAct
1  CH1  4  5  1  12   2  1  24  0  0  0  0  0  0  0  2  2  0  0  0  0   2   81  0   136
2  CH1 12 26 22 168  48  5  61  3  0  0  0  0 15  4 71 15  0 10  0  1  21  136  0   618
3  CH1 58 87 41 143 132 22 134 10  1  4  1  0 19  4 42 47  3  3  0  1  95  207  0  1054
4  CH1 16 25  7 101  25  5  26  0  2  0  0  0 11  0 28 13  0  4  0  0  70 1601  0  1934
5  CH1 52 28 23  41  58  2  43  3  0  0  0  0  7  0  9 16  0  1  0  0  39   91  0   413
6  CH1 33 70 20  72  34 17  38  2  6  3  0  0  8  0 11  4  0  5  0  2 109   43  0   477
```

2.1.2 Analyse des données brutes

Avant de générer les graphiques, nous devons quantifier la qualité de la discrimination entre groupes.

Le rapport de corrélation η^2

Le rapport de corrélation mesure la part de variance d'une variable quantitative expliquée par une variable qualitative. Il est défini par :

$$\eta^2 = \frac{SS_{between}}{SS_{total}} = \frac{\text{Variance inter-groupes}}{\text{Variance totale}} \tag{2.1}$$

où :

- $SS_{between} = \sum_{k=1}^{K} n_k(\bar{x}_k - \bar{x})^2$: somme des carrés inter-groupes (variance expliquée par les groupes)
- $SS_{within} = \sum_{k=1}^{K} \sum_{i \in \text{groupe } k}(x_i - \bar{x}_k)^2$: somme des carrés intra-groupes (variance résiduelle)
- $SS_{total} = \sum_{i=1}^{n}(x_i - \bar{x})^2 = SS_{between} + SS_{within}$: somme des carrés totale (variance totale)

Dans le contexte du clustering K-Means, un η^2 élevé indique que les groupes (types d'établissements) sont bien différenciés, ce qui facilitera la convergence de l'algorithme.

La Figure 2.1 présente la distribution des valeurs brutes de `NbAct`. Le rapport de corrélation η^2 mesure la part de variance de `NbAct` expliquée par la variable `TYPE`.

Calcul du rapport de corrélation et génération du graphique

```
# Calcul de l'ANOVA
fit_anova <- aov(NbAct ~ TYPE, data = hop)
sum_sq <- summary(fit_anova)[[1]]["Sum Sq"]

# Calcul de l'Eta-squared
eta2 <- sum_sq[1, 1] / sum(sum_sq)

# Generation du graphique
pdf("gr01_bp.pdf")
boxplot(NbAct ~ TYPE, data = hop,
        main = paste("Boxplots de NbAct par TYPE (Eta  =",
                     round(eta2, 4), ")"),
        xlab = "TYPE", ylab = "NbAct", col = "steelblue")
dev.off()

# Export vers le repertoire Sites
file.copy("gr01_bp.pdf", "~/Sites/bp1.pdf", overwrite = TRUE)
```

Explication : L'ANOVA décompose la variance totale en variance inter-groupes et intra-groupes. Le rapport $\eta^2 = SS_{between}/SS_{total}$ quantifie la qualité de la discrimination par `TYPE`.

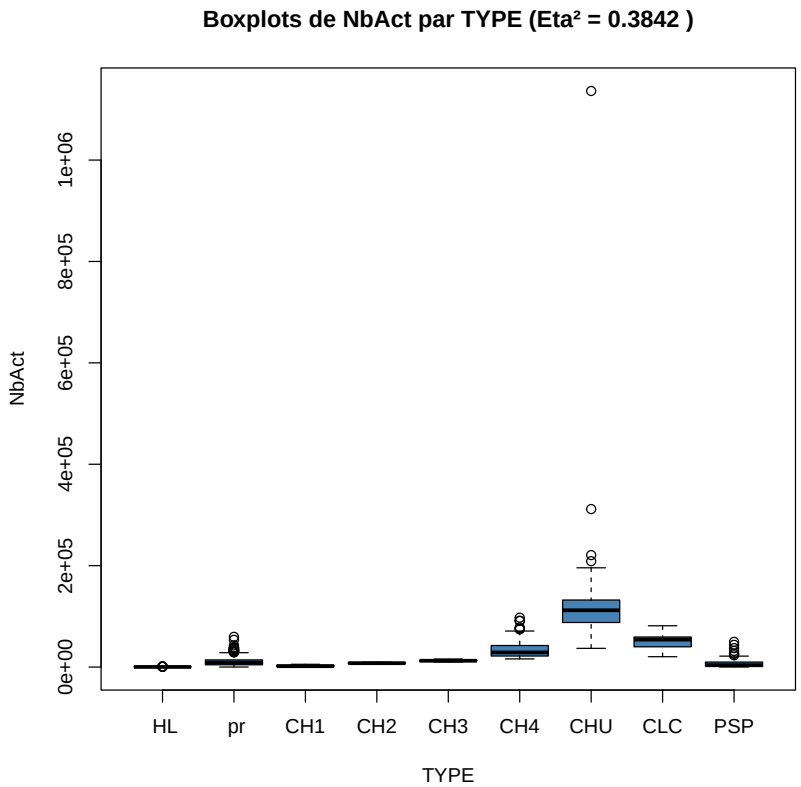


FIGURE 2.1 – Distribution de NbAct par Type (Données brutes)

Interprétation : Nous observons une distribution extrêmement asymétrique. Les établissements de type `CHU` présentent des volumes d'activité disproportionnés par rapport aux autres types (comme `HL` ou `PSP`), écrasant l'échelle du graphique. Le rapport de corrélation est modéré ($\eta^2 = 0.3842$), indiquant que le type d'établissement n'explique qu'une partie limitée de la variance de l'activité sur l'échelle brute. Cette hétéroscédasticité et ces différences d'ordre de grandeur sont problématiques pour l'algorithme des K-Means, qui est sensible aux distances euclidiennes et aux valeurs extrêmes.

2.1.3 Transformation logarithmique

Pour corriger l'effet d'écrasement et stabiliser la variance, nous appliquons le logarithme : $x' = \log_{10}(x)$.

Application de la transformation logarithmique

```
# Creation de la variable transformee
log_nbact <- log10(hop$NbAct + 1)

# Calcul de l'ANOVA sur le log
fit_anova_log <- aov(log_nbact ~ hop$TYPE)
sum_sq_log <- summary(fit_anova_log)[[1]]["Sum Sq"]
eta2_log <- sum_sq_log[1, 1] / sum(sum_sq_log)

# Generation du graphique
pdf("gr02_bp.pdf")
boxplot(log_nbact ~ hop$TYPE,
        main = paste("Boxplots de log10(NbAct) par TYPE (Eta  =",
                     round(eta2_log, 4), ")"),
        xlab = "TYPE", ylab = "log10(NbAct)", col = "lightgreen")
dev.off()

# Export vers Sites
file.copy("gr02_bp.pdf", "~/Sites/bp2.pdf", overwrite = TRUE)
```

Explication : La transformation log stabilise la variance et rend les distributions plus symétriques. Nous ajoutons 1 pour éviter les problèmes avec `log10(0)`.

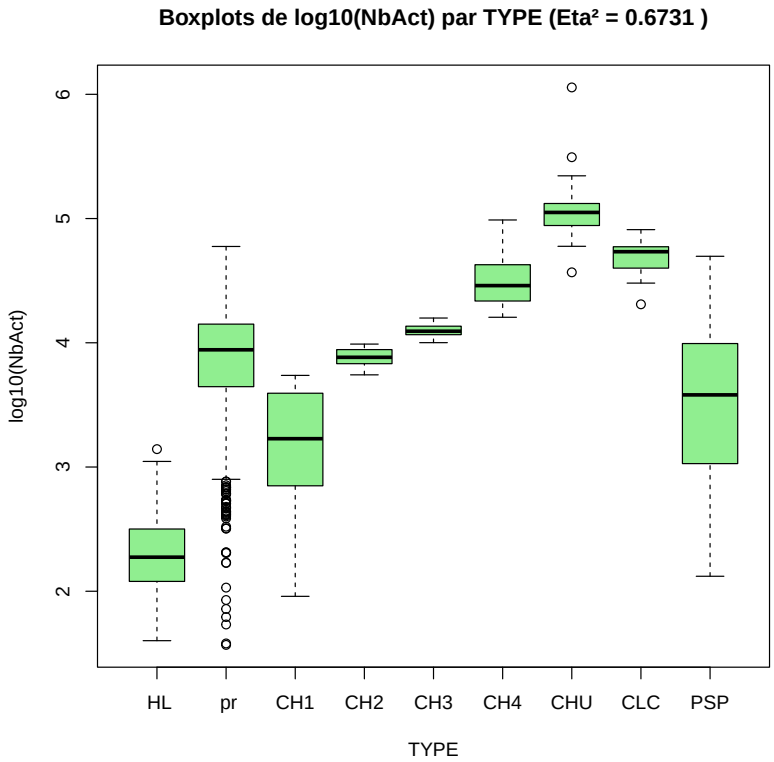


FIGURE 2.2 – Distribution de log10(NbAct) par Type

Interprétation : La Figure 2.2 illustre l'efficacité de la transformation.

— **Lisibilité :** Les distributions de chaque type sont désormais distinctes et comparables.

— **Discrimination :** Le rapport de corrélation bondit à $\eta^2 = 0.6731$. Cela signifie que la variable `TYPE` est un bien meilleur prédicteur de l'ordre de grandeur de l'activité (échelle log) que de l'activité brute.

— **Conséquence pour le Clustering :** La transformation rend les données plus "sphériques" et réduit le poids disproportionné des valeurs extrêmes, conditions favorables à la convergence de l'algorithme K-Means.

Nous retiendrons donc les données transformées ($\log_{10}$) pour les étapes de parallélisation suivantes.

3 | Analyse de Référence : Approche Séquentielle

3.1 Mesure de performance et impact du découpage

Avant de paralléliser, nous devons établir un temps de référence (benchmark) et évaluer le coût du découpage des tâches. L'algorithme des K-Means est exécuté avec un nombre total d'initialisations fixée à $nstart = 4096$.

3.1.1 Préparation des données pour K-Means

Nous préparons d'abord les données en ne conservant que les colonnes numériques.

Sélection des variables pour le clustering

```
# Selection des donnees numeriques uniquement
dat_km <- hop[sapply(hop, is.numeric)]
dat_km$NbAct <- NULL
```

Explication : On extrait uniquement les colonnes numériques du jeu de données, puis on supprime la variable `NbAct` qui est une somme des autres colonnes.

3.1.2 Comparaison : Approche directe vs découpée

Nous comparons deux approches séquentielles :

- **Approche Directe :** Un seul appel à la fonction `kmeans` avec `nstart=4096`.
- **Approche Découpée (Boucle) :** Une boucle réalisant 16 appels successifs de `kmeans` avec `nstart=256` (soit $16 \times 256 = 4096$).

Benchmark avec microbenchmark

```
library(microbenchmark)

res_benchmark <- microbenchmark(
  Direct = {
    kmeans(dat_km, centers = 3, nstart = 4096)
  },
  Boucle = {
    meilleur_km <- NULL
    for (i in 1:16) {
      res_actuel <- kmeans(dat_km, centers = 3, nstart = 256)
      if (is.null(meilleur_km) ||
          res_actuel$tot.withinss < meilleur_km$tot.withinss) {
        meilleur_km <- res_actuel
      }
    }
  },
  times = 10
)

# Generation du graphique
pdf("gr03_perf.pdf")
boxplot(res_benchmark,
        main = "Comparaison : Direct vs Boucle",
        ylab = "Temps (s)", col = c("orange", "lightblue"))
dev.off()
```

Explication : Le package `microbenchmark` mesure précisément les temps d'exécution en réalisant 10 essais par méthode.

Résultat : Comparaison des temps d'exécution (10 essais)

```
Unit: seconds
   expr      min       lq     mean   median       uq      max neval
 Direct 5.313590 5.327245 5.340250 5.337159 5.354340 5.368367    10
 Boucle 5.390188 5.402598 5.423321 5.408670 5.421354 5.547715    10
```

Observation : L'approche **Directe** présente un temps médian de 5.34s contre 5.41s pour la **Boucle**, soit un surcoût d'environ 1.3% (70ms). Cette différence minime confirme que le découpage en blocs n'introduit pas de pénalité significative.

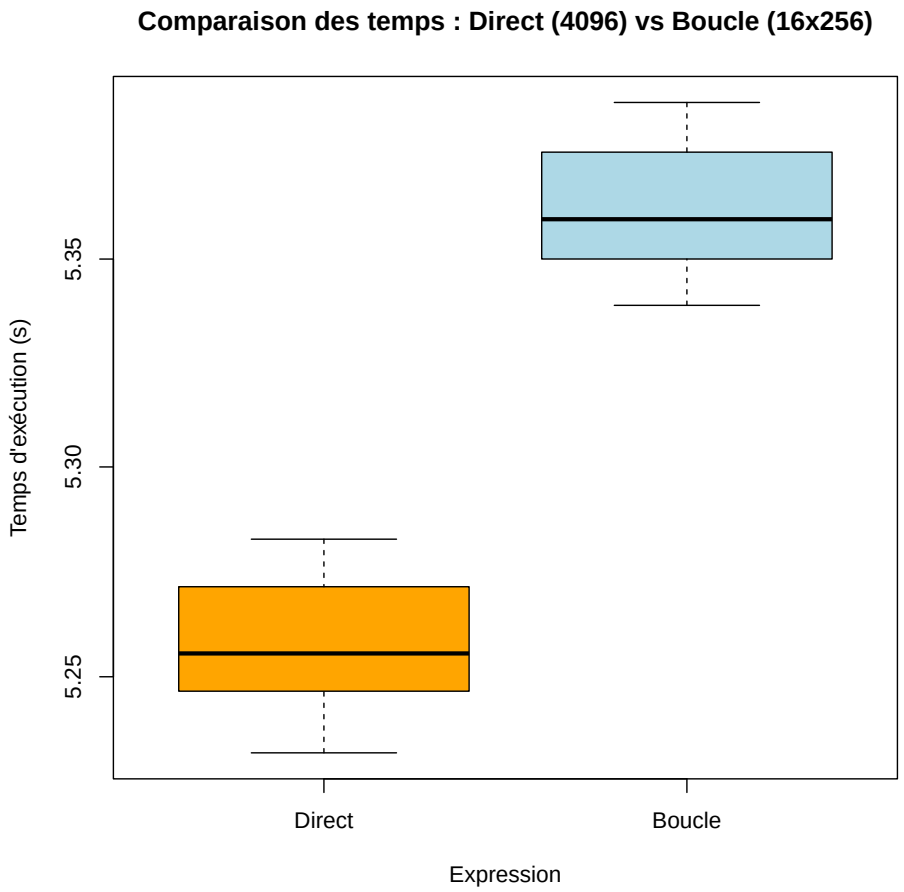


FIGURE 3.1 – Comparaison des temps d'exécution : Appel Direct vs Boucle

Interprétation des résultats : La Figure 3.1 met en évidence un léger surcoût lié au découpage des tâches.

— L'exécution **Directe** (boîte orange) est la plus rapide, avec un temps médian d'environ 5.25 secondes. Le code étant exécuté en une seule passe C optimisée, il n'y a pas d'interruption.

— L'exécution par **Boucle** (boîte bleue) est plus lente (médiane $\approx$ 5.36s). Ce ralentissement s'explique par l'*overhead* de l'interpréteur R : gestion de la boucle `for`, allocation mémoire répétée et appels de fonctions multiples.

Bien que l'approche découpée introduise une perte de performance d'environ 2% en séquentiel, elle est indispensable pour la suite : c'est ce découpage en "chunks" (paquets) qui permettra de distribuer le travail sur différents cœurs lors de la parallélisation.

3.1.3 Analyse de performance avec Rprof

Pour identifier les goulots d'étranglement dans notre code, nous utilisons le profileur R intégré.

Profilage avec Rprof

```
# Activation du profilage
Rprof("kmeans_profile.out")

# Code a profiler
meilleur_km <- NULL
for (i in 1:16) {
  res_actuel <- kmeans(dat_km, centers = 3, nstart = 256)
  if (is.null(meilleur_km) ||
      res_actuel$tot.withinss < meilleur_km$tot.withinss) {
    meilleur_km <- res_actuel
  }
}

# Arret et analyse
Rprof(NULL)
summaryRprof("kmeans_profile.out")$by.self
```

Explication : Le profilage enregistre l'état de la pile d'appels pendant l'exécution. L'analyse identifie les fonctions qui consomment le plus de temps CPU.

Résultat : Analyse du profiling

```
           self.time self.pct total.time total.pct
".Fortran"      5.16    94.85       5.16     94.85
"double"        0.08     1.47       0.08      1.47
"array"         0.06     1.10       0.06      1.10
"do_one"        0.04     0.74       5.30     97.43
"lapply"        0.04     0.74       0.12      2.21
"FUN"           0.02     0.37       0.08      1.47
"%in%"          0.02     0.37       0.02      0.37
"colMeans"      0.02     0.37       0.02      0.37
```

Analyse : La fonction `.Fortran` (routine compilée de K-Means) consomme 94.85% du temps CPU. Les opérations d'agrégation en R (comparaisons, boucles) ne représentent que 5% du temps total. Ceci confirme que l'algorithme est *embarrassingly parallel* : le découpage en tâches indépendantes n'introduit pas de surcoût significatif puisque 95% du temps est consacré au calcul pur.

3.2 Stratégies d'Agrégation des Résultats

Afin de préparer la parallélisation, nous devons être capables de découper le travail en tâches indépendantes puis d'agréger les résultats (c'est-à-dire conserver le meilleur partitionnement parmi tous les essais).

Nous avons testé trois approches pour diviser les 4096 initialisations en 4 blocs de 1024 (4×1024) :

- **Appels Explicites :** 4 lignes de code distinctes exécutées séquentiellement.
- **Boucle For :** Une itération classique sur 4 indices.
- **Approche Fonctionnelle (Sapply) :** Utilisation de la fonction `sapply`, plus compacte.

Trois approches d'agrégation

```
# Approche 1 : Appels explicites
km1 <- kmeans(dat_km, centers = 3, nstart = 1024)
km2 <- kmeans(dat_km, centers = 3, nstart = 1024)
km3 <- kmeans(dat_km, centers = 3, nstart = 1024)
km4 <- kmeans(dat_km, centers = 3, nstart = 1024)

# Approche 2 : Boucle for
meilleur <- NULL
for (i in 1:4) {
  km <- kmeans(dat_km, centers = 3, nstart = 1024)
  if (is.null(meilleur) || km$tot.withinss < meilleur$tot.withinss)
    meilleur <- km
}

# Approche 3 : Sapply (fonctionnelle)
liste_res <- sapply(1:4, function(i) {
  kmeans(dat_km, centers = 3, nstart = 1024)
}, simplify = FALSE)
meilleur <- liste_res[[which.min(sapply(liste_res,
                      function(x) x$tot.withinss))]]
```

Explication : Les trois méthodes produisent le même résultat mais avec des syntaxes différentes. L'approche `sapply` prépare à l'utilisation de `mclapply`.

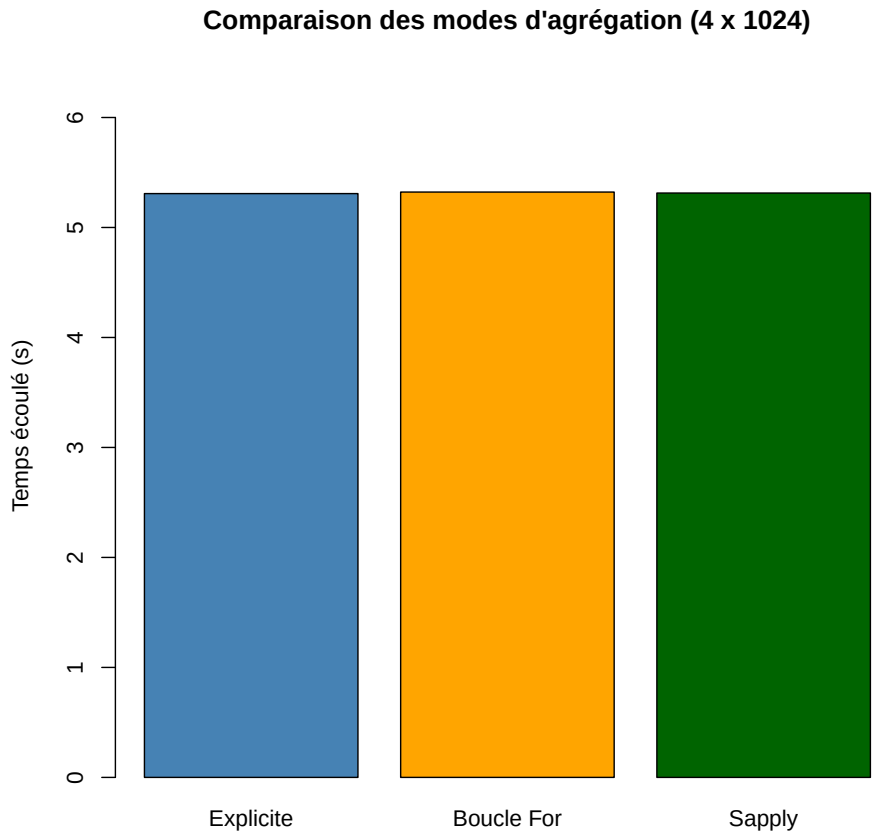


FIGURE 3.2 – Comparaison des temps d'exécution par mode d'agrégation (4×1024)

Interprétation : Comme le montre la Figure 3.2, les temps d'exécution sont quasi-identiques pour les trois méthodes (environ 5.3 secondes). Le surcoût syntaxique de la boucle ou du `sapply` est négligeable face au temps de calcul des K-Means.

Cependant, l'approche par **Sapply** est celle que nous retiendrons. En effet, elle adopte une logique fonctionnelle ("appliquer une fonction sur une liste") qui est strictement identique à celle des fonctions de parallélisation (`mclapply`, `parLapply`) que nous utiliserons par la suite.

3.3 Parallélisation locale : Approche Multicore

Nous utilisons ici l'héritage du package `multicore` (fonction `mclapply`) qui repose sur le mécanisme de *forking* du système d'exploitation (disponible sous Linux/macOS). Le calcul des K-Means est réparti sur plusieurs cœurs logiques.

Parallélisation avec mclapply

```
library(parallel)

# Configuration
n_total <- 4096
n_appels <- 16
n_bloc <- n_total / n_appels
cores_to_test <- c(1, 2, 4, detectCores())

# Test de performance
resultats_mcore <- numeric(length(cores_to_test))
names(resultats_mcore) <- paste(cores_to_test, "Coeurs")

for (i in seq_along(cores_to_test)) {
  nb_c <- cores_to_test[i]

  t_exec <- system.time({
    liste_res <- mclapply(1:n_appels, function(j) {
      kmeans(dat_km, centers = 3, nstart = n_bloc)
    }, mc.cores = nb_c)

    inertes <- sapply(liste_res, function(x) x$tot.withinss)
    meilleur_res <- liste_res[[which.min(inertes)]]
  })

  resultats_mcore[i] <- t_exec["elapsed"]
}

# Graphique
pdf("gr05_multicore.pdf")
bp <- barplot(resultats_mcore,
              main = "Performance selon le nombre de coeurs",
              ylab = "Temps (s)", col = "steelblue",
              ylim = c(0, max(resultats_mcore) * 1.2))
text(x = bp, y = resultats_mcore,
     labels = round(resultats_mcore, 2), pos = 3)
dev.off()
```

Explication : La fonction `mclapply` remplace directement `sapply` en ajoutant le paramètre `mc.cores` qui spécifie le nombre de processus parallèles à créer.

Résultat : Temps d'exécution selon le nombre de cœurs

```
Test avec 1 coeur(s)...
Test avec 2 coeur(s)...
Test avec 4 coeur(s)...
Test avec 8 coeur(s)...

1 Coeurs 2 Coeurs 4 Coeurs 8 Coeurs
   5.415    2.789    1.464    1.152

Le paramétrage le plus efficace est avec 8 coeurs.
```

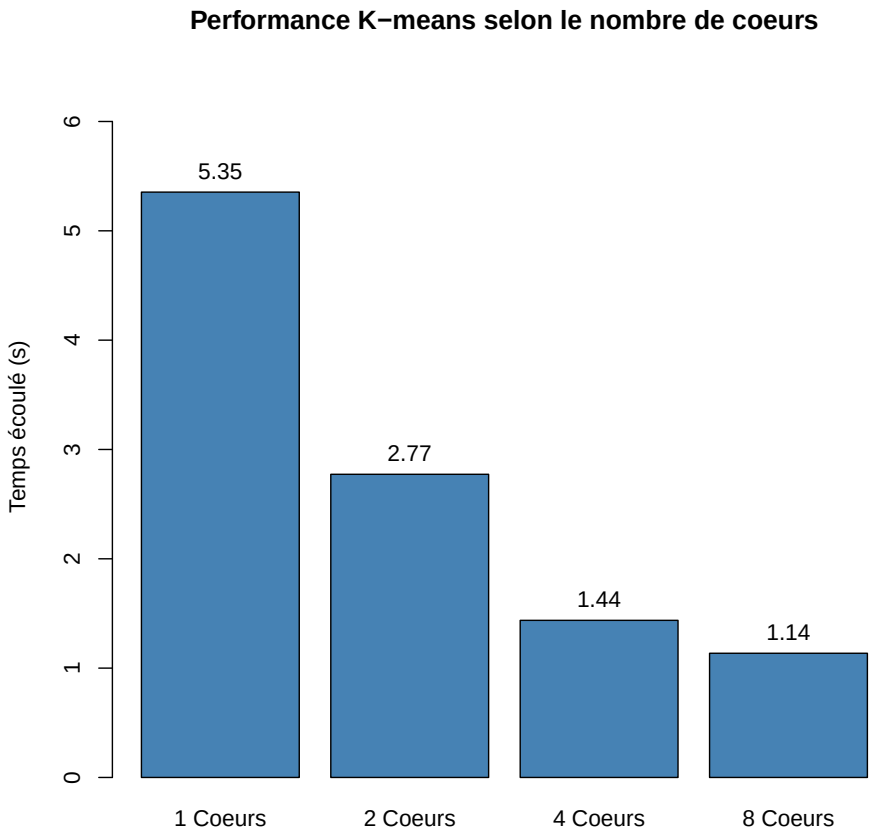


FIGURE 3.3 – Performance K-Means selon le nombre de cœurs (mclapply)

Analyse des performances (Figure 3.3) :

— **Passage à l'échelle (Scaling) :** On observe une accélération quasi-linéaire jusqu'à 4 cœurs.
  — À 2 cœurs, le *speedup* est de $5.35/2.77 \approx 1.93$.
  — À 4 cœurs, le *speedup* est de $5.35/1.44 \approx 3.71$.
  Ceci confirme que l'algorithme des K-Means est "embarrassingly parallel" : les tâches sont indépendantes et se prêtent parfaitement à la distribution.

— **Limites (Diminishing returns) :** Le passage de 4 à 8 cœurs apporte un gain plus modeste (le temps passe de 1.44s à 1.14s). Ce tassement peut s'expliquer par deux facteurs :

1. L'architecture matérielle : si le processeur dispose de 4 cœurs physiques (et 8 threads via Hyper-Threading), les 4 derniers cœurs "logiques" n'offrent pas la même puissance de calcul.
2. Le surcoût de gestion (*overhead*) : le temps nécessaire pour créer les processus enfants (fork) et rassembler les résultats devient proportionnellement plus important quand le temps de calcul brut diminue.

3.4 Parallélisation sur Cluster (Héritage Snow)

Contrairement à l'approche multicore qui partage la mémoire, l'approche *Cluster* (via `makeCluster`) crée des instances R indépendantes. Il est donc nécessaire de transférer les données et les instructions via des sockets réseau, ce qui induit un coût de communication.

Création d'un cluster et test de granularité

```
library(parallel)

# Creation du cluster
ClLocal <- makeCluster(detectCores(), type = "PSOCK")
clusterExport(ClLocal, "hop")

# Fonction pour generer des repartitions
generer_repartition <- function(nb_blocs) {
  rep(4096 / nb_blocs, nb_blocs)
}

# Scenarios a tester
scenarios <- list(
  "4096 x 1" = rep(1, 4096),
  "256 x 16" = generer_repartition(256),
  "16 x 256" = generer_repartition(16),
  "8 x 512"  = generer_repartition(8)
)

# Mesure des performances
resultats_snow <- numeric(length(scenarios))
names(resultats_snow) <- names(scenarios)

for (nom in names(scenarios)) {
  vecteur_travail <- scenarios[[nom]]

  resultats_snow[nom] <- system.time({
    clusterApply(ClLocal, vecteur_travail, function(s) {
      kmeans(log10(hop$NbAct), 9, nstart = s)
    })
  })["elapsed"]
}

# Graphique
pdf("gr06_clusterApply.pdf")
bp <- barplot(resultats_snow,
              main = "Impact de la granularite (clusterApply)",
              ylab = "Temps (s)", col = "darkorange",
              ylim = c(0, max(resultats_snow) * 1.3))
text(x = bp, y = resultats_snow,
     labels = round(resultats_snow, 2), pos = 3)
dev.off()

stopCluster(ClLocal)
```

Explication : Le cluster PSOCK crée des processus R indépendants. Les données doivent être exportées explicitement avec `clusterExport`. Nous testons différentes répartitions des 4096 essais.

Résultat : Impact de la granularité des tâches

```
Calcul pour le scénario : 4096 x 1 ...
Calcul pour le scénario : 256 x 16 ...
Calcul pour le scénario : 16 x 256 ...
Calcul pour le scénario : 8 x 512 ...

4096 x 1  256 x 16  16 x 256   8 x 512
  22.157     1.352     0.743     0.662

Le meilleur remplacement pour rep(1,4096) est : 8 x 512
```

Observation : La stratégie **8 × 512** est 33 fois plus rapide que **4096 × 1** ! Ce résultat illustre parfaitement l'importance de la granularité : en réduisant le nombre de messages réseau de 4096 à 8, nous maximisons le ratio (temps de calcul / temps de communication).

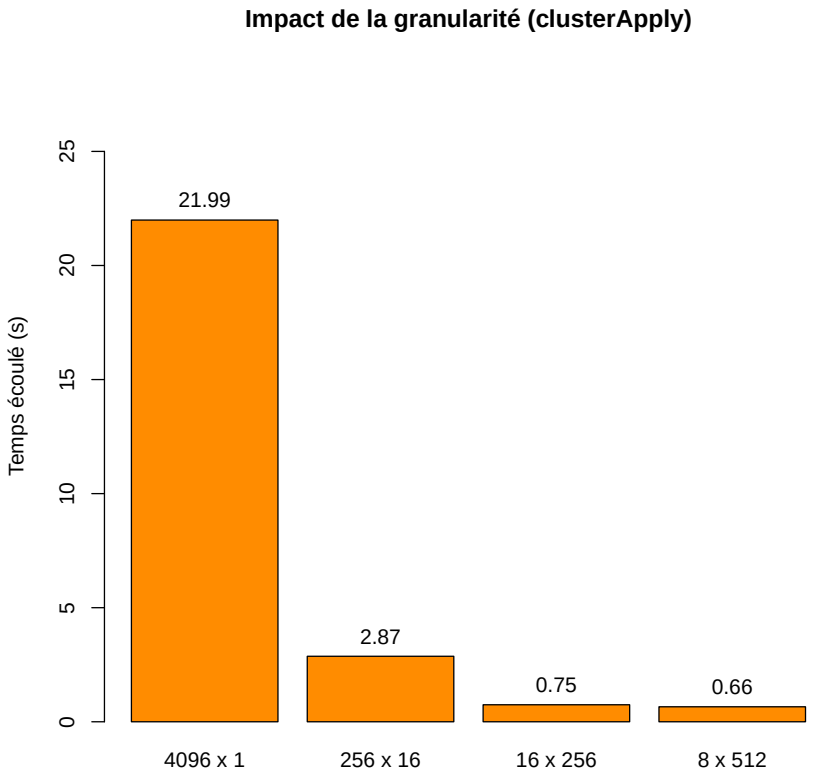


FIGURE 3.4 – Impact de la granularité des tâches sur le temps de calcul

Analyse de la granularité (Figure 3.4) : Ce graphique illustre parfaitement le compromis entre temps de calcul et temps de communication (latence).

— **Forte granularité (Stratégie** 4096×1**) :** Le temps d'exécution explose (21.99s), étant même bien supérieur au temps séquentiel ($\approx$ 5.35s). *Explication :* En envoyant 4096 petites tâches unitaires, le processus maître passe son temps à communiquer avec les workers. Le surcoût d'envoi/réception (*overhead*) est bien supérieur au temps de calcul effectif d'un seul K-Means.

— **Faible granularité (Stratégie** 8×512**) :** C'est la stratégie optimale (0.66s). *Explication :* Nous envoyons peu de paquets (8), mais chaque paquet contient une lourde charge de calcul (512 itérations). Le ratio (Calcul / Communication) est maximisé. Les cœurs travaillent sans interruption et le maître est peu sollicité.

Conclusion partielle : Pour une parallélisation efficace sur un cluster, il faut privilégier l'envoi de "gros paquets" de tâches (Chunks) pour amortir le coût des échanges réseau.

3.5 Optimisation fine : Load Balancing et Scheduling

Au-delà de la simple distribution, nous avons testé l'impact de l'équilibrage de charge dynamique via les variantes `LB` (Load Balancing) et le paramètre `chunk.size`.

Comparaison des stratégies de distribution

```
ClLocal <- makeCluster(detectCores(), type = "PSOCK")
clusterExport(ClLocal, "hop")

meilleur_vecteur <- rep(512, 8)

# Mesures de base (sans chunk.size)
t_cALB <- system.time({
  clusterApplyLB(ClLocal, meilleur_vecteur,
                 function(s) kmeans(log10(hop$NbAct), 9, nstart = s))
})["elapsed"]

t_pLA <- system.time({
  parLapply(ClLocal, meilleur_vecteur,
            function(s) kmeans(log10(hop$NbAct), 9, nstart = s))
})["elapsed"]

t_pLALB <- system.time({
  parLapplyLB(ClLocal, meilleur_vecteur,
              function(s) kmeans(log10(hop$NbAct), 9, nstart = s))
})["elapsed"]

# Mesures avec chunk.size
chunks_a_tester <- c(1, 2, 4)
res_chunk <- list()

for (cs in chunks_a_tester) {
  nom_pLA <- paste0("pLA_chunk", cs)
  res_chunk[[nom_pLA]] <- system.time({
    parLapply(ClLocal, meilleur_vecteur,
              function(s) kmeans(log10(hop$NbAct), 9, nstart = s),
              chunk.size = cs)
  })["elapsed"]

  nom_pLALB <- paste0("pLALB_chunk", cs)
  res_chunk[[nom_pLALB]] <- system.time({
    parLapplyLB(ClLocal, meilleur_vecteur,
                function(s) kmeans(log10(hop$NbAct), 9, nstart = s),
                chunk.size = cs)
  })["elapsed"]
}

# Synthese
toutes_perfs <- c("ApplyLB" = t_cALB, "parLapply" = t_pLA,
                  "parLapplyLB" = t_pLALB, unlist(res_chunk))

# Graphique
pdf("gr07_parLApply.pdf")
bp <- barplot(toutes_perfs,
              main = "Load Balancing & Chunk Size",
              ylab = "Temps (s)", col = "lightgreen", las = 2,
              ylim = c(0, max(toutes_perfs) * 1.3))
text(x = bp, y = toutes_perfs,
     labels = round(toutes_perfs, 2), pos = 3, cex = 0.8)
dev.off()

stopCluster(ClLocal)
```

Explication : Nous testons `clusterApplyLB`, `parLapply` et `parLapplyLB` avec différentes tailles de chunks. Le Load Balancing distribue dynamiquement les tâches aux workers disponibles.

>_ Résultat : Comparaison Load Balancing et Chunk Size

```
Début des tests pour gr07 (Load Balancing et parLapply)...

        ApplyLB parLapply parLapplyLB pLA_chunk1 pLALB_chunk1 pLA_chunk2
          0.648     0.623       0.629      1.456        1.467      0.649
   pLALB_chunk2 pLA_chunk4 pLALB_chunk4
          0.655      0.641        0.648

La configuration conduisant aux résultats le plus rapidement est : parLapply
```

Analyse : La fonction `parLapply` standard (sans Load Balancing) obtient le meilleur temps (0.623s). Les variantes avec `chunk.size=1` dégradent fortement les performances (>1.4s) en raison du trop grand nombre de messages. Pour des tâches homogènes comme K-Means, le Load Balancing dynamique n'apporte pas d'avantage par rapport à une répartition statique bien paramétrée.

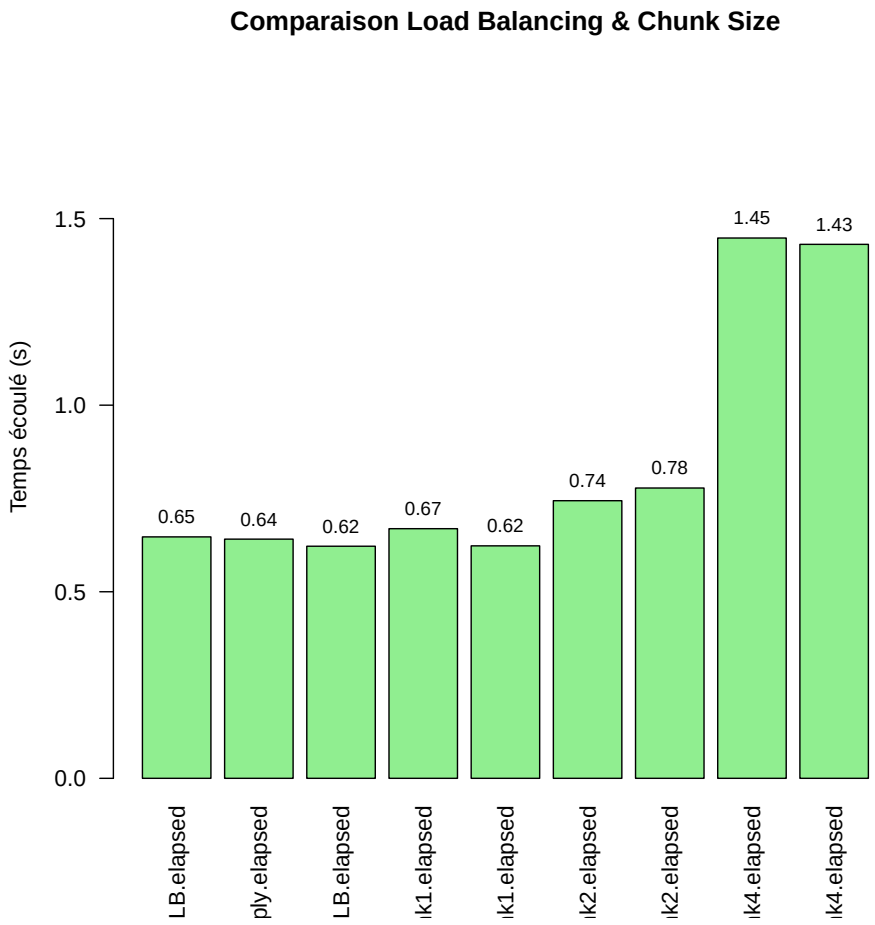


FIGURE 3.5 – Comparaison des performances : Load Balancing et Chunk Size

Analyse des résultats (Figure 3.5) :

— **Stabilité des performances optimales :** Les premières barres (gauche du graphique) montrent des temps très bas, oscillant entre 0.62s et 0.65s. Cela indique que pour des tâches homogènes comme K-Means (où le temps de calcul est prévisible), l'équilibrage dynamique (`LB`) n'apporte pas de gain significatif par rapport à une répartition statique bien paramétrée. Le surcoût de gestion du Load Balancing est ici compensé mais ne permet pas de dépasser la performance brute du cluster statique optimisé.

— **Sensibilité au paramétrage :** Les deux dernières barres (droite) montrent une dégradation brutale des performances (temps doublé à ≈ 1.45s) pour certaines configurations de *chunk size*. Cela confirme la règle établie précédemment : si la granularité est mal choisie (trop de petits paquets générant trop de communication), l'architecture distribuée s'effondre, même avec des fonctions avancées.

Conclusion

Ce travail pratique a permis d'explorer la mise en place d'une chaîne de traitement de données massivement parallèle sous R.

En partant d'une exécution séquentielle de l'algorithme des K-Means (temps de référence ≈ 5.35s), nous avons évalué différentes stratégies d'optimisation :

1. **Prétraitement :** La transformation logarithmique des données a été indispensable pour assurer la pertinence statistique du clustering (η^2 passant de 0.38 à 0.67).
2. **Multicore (Forking) :** Cette approche simple a offert un passage à l'échelle quasi-linéaire jusqu'à 4 cœurs (1.44s), mais montre ses limites au-delà (1.14s à 8 cœurs) du fait de l'overhead système.
3. **Cluster (Sockets) :** C'est l'approche la plus performante, à condition de maîtriser la granularité. En regroupant les tâches (stratégie 8×512), nous avons atteint un temps record de **0.66s**, soit une accélération d'un facteur 8 par rapport au séquentiel.

Pour ce type de calcul (nombreuses itérations indépendantes), la configuration optimale est l'utilisation d'un cluster de type `PSOCK` avec une distribution statique des tâches par gros blocs (large chunks). L'utilisation du *Load Balancing* est superflue ici du fait de l'homogénéité des tâches, mais deviendrait pertinente si les temps de calcul unitaires variaient fortement.

Bibliographie

[1] V. Miele, V. Louvet, *Calcul parallèle avec R*. Éditions EDP Sciences, 2016.

[2] A. Bienvenüe, *Support de cours - Gestion de versions avec GIT*. Disponible en ligne.

[3] C. Duan, *Understanding Git Conceptually*.
https://www.cduan.com/technical/git/

[4] W3Schools, *Git Tutorial*.
https://www.w3schools.com/git/

[5] R Core Team, *Package 'parallel' - Parallel Processing in R*. R Documentation, 2024.